Creating a Custom Docker Image

In this train, we'll learn the basics of creating your own Docker images using a Dockerfile.

Learning Objectives

By the end of this train you should be able to:

- Create your own Docker images using a Dockerfile.

Why Create a Custom Docker Image?

- **Faster Job Execution:** Packaging all necessary tools and dependencies reduces setup time.
- **Cleaner Configuration:** A custom image simplifies configuration files by eliminating lengthy installation scripts.

> **Prerequisites:**
> - A working Docker installation (Install Docker).
> - Basic knowledge of Docker commands.

How to Create a Custom Docker Image

1. Creating a Dockerfile

A Dockerfile is a text file that contains a list of instructions for building a Docker image.

- Create a new Dockerfile in your project folder:
  `touch Dockerfile`

2: Choosing a Base Image

Select an official image from Docker Hub as the foundation for your custom image. Add the following line to your Dockerfile:

- `FROM alpine:3.5`

3. Installing Additional Tools

Use the `RUN` instruction to install necessary packages:

- `RUN apk add --no-cache bash git openssh tar gzip ca-certificates`

4: Adding Files and Directories

To include additional files in the image, use the `ADD` instruction:

- `ADD ./workdir/contacts /usr/bin/contacts`

5: Defining an Entrypoint

The `ENTRYPOINT` instruction specifies the default executable when the container starts:

- `ENTRYPOINT ["contacts"]`

6: Building the Image

Run the following command to build the image:

- `docker build -t my-custom-image:1.0 .`

7: Storing the Image in a Registry

To make the image available for use, store it in Docker Hub or another registry:

- Log in to Docker Hub:
  `docker login`
- Tag the image:
  `docker tag my-custom-image:1.0 myusername/my-custom-image:1.0`
- Push the image to Docker Hub:
  `docker push myusername/my-custom-image:1.0`

8: Using the Custom Image

Now that the image is stored in a registry, you can use it in any project:

- `docker run -d myusername/my-custom-image:1.0`

> **Find a practical example** of a detailed custom Dockerfile example for Ruby here.

Conclusion

Creating custom Docker images allows for optimized and portable application environments. By defining a Dockerfile, you can streamline deployments and ensure consistency across development and production environments.

Key Takeaways

- **Takeaway 1:** A **Dockerfile** contains instructions to build a custom image.
- **Takeaway 2:** The `FROM` instruction sets the base image.
- **Takeaway 3:** The `RUN` instruction installs dependencies.

- **Takeaway 4:** The `ENTRYPOINT` defines the default command executed in the container.
- **Takeaway 5: Custom images** can be pushed to Docker Hub and used in any environment.